TRƯỜNG ĐẠI HỌC CỬU LONG • KHOA CNTT – TRÍ TUỆ NHÂN TẠO

BÀI GIẢNG KIỂM THỬ PHẦN MỀM

CHƯƠNG 6

SELENIUM

Tổng quan – Selenium IDE – Selenium Grid – Selenium WebDriver (C#) – Xây dựng kịch bản kiểm thử

Thời lượng: **5 tiết** (1 tiết = 45 phút)

Biên soạn: ThS. Lôi Thị Tú Trân

Vĩnh Long – 2026

Mục tiêu chương học

Kiến thức

Trình bày được khái niệm, đặc điểm, ưu điểm và hạn chế của Selenium.

Kiến thức

Phân biệt được vai trò của Selenium IDE, WebDriver và Grid.

Kiến thức

Giải thích được các khái niệm Session, Element, Locator, Wait, Alert.

Kỹ năng

Ghi và phát lại được kịch bản kiểm thử cơ bản bằng Selenium IDE.

Kỹ năng

Viết được chương trình Selenium WebDriver bằng C# để tự động hoá thao tác web.

Kỹ năng

Xây dựng, thực thi và bảo trì được kịch bản kiểm thử tự động cho một chức năng web.

Tài liệu tham khảo: **Selenium Documentation** – https://www.selenium.dev/documentation/

Nội dung chương 6

- **6.1** Tổng quan về Selenium: khái niệm, đặc điểm, ưu điểm và hạn chế
- **6.2** Các thành phần của bộ công cụ Selenium
- **6.3** Selenium IDE: cài đặt, giao diện, ghi – phát lại, các lệnh thông dụng
- **6.4** Selenium Grid: kiến trúc Hub – Node và thực thi song song
- **6.5** Selenium WebDriver: khái niệm, kiến trúc, các khái niệm cốt lõi
- **6.6** Các thao tác cơ bản trong Selenium WebDriver với C#
- **6.7** Quy trình xây dựng kịch bản kiểm thử với Selenium
- **6.8** Bài tập vận dụng + Bài tập tự luyện + Câu hỏi ôn tập

THỜI LƯỢNG: ≈ 0,75 TIẾT

6.1

Tổng quan về Selenium

- Khái niệm
- Tình huống thực tế
- Đặc điểm
- Ưu điểm và hạn chế

Selenium là gì?

Selenium là một **bộ công cụ kiểm thử tự động mã nguồn mở (open-source test automation tool)** dành cho ứng dụng web, hỗ trợ hoạt động trên nhiều trình duyệt và nhiều nền tảng khác nhau như Windows, macOS, Linux.

Tự động hoá trình duyệt

Cho phép tester tự động thực hiện các thao tác: **click chuột, nhập dữ liệu, chuyển trang và kiểm tra kết quả hiển thị**.

Hỗ trợ đa ngôn ngữ

Viết kịch bản bằng **C#, Java, Python, Ruby, JavaScript, PHP** ⇒ dễ tích hợp vào nhiều môi trường phát triển.

Dùng nhiều cho hồi quy

Nhờ khả năng thực hiện thao tác lặp lại nhanh và chính xác, Selenium được dùng chủ yếu cho **kiểm thử chức năng web và kiểm thử hồi quy**.

Selenium **miễn phí hoàn toàn**, không mất chi phí bản quyền – đặc biệt phù hợp với môi trường học tập và các dự án có ngân sách hạn chế.

Tình huống thực tế: nhóm sinh viên kiểm thử website bán hàng

BỐI CẢNH

- Một nhóm sinh viên được giao kiểm thử website bán hàng trực tuyến có các chức năng: **đăng nhập, tìm kiếm sản phẩm, đặt hàng**
- Ban đầu nhóm kiểm thử thủ công, nhưng **số lượng test case lớn** và **cần kiểm thử lặp lại nhiều lần** ⇒ mất nhiều thời gian, dễ sai sót
- Nhóm quyết định dùng **Selenium**, viết kịch bản bằng **C#** và chạy trên nhiều trình duyệt khác nhau

NHỮNG ĐIỀU NHÓM NHẬN THẤY

- ✔ Không cần mua phần mềm kiểm thử • ✔ Dễ tìm tài liệu khi gặp lỗi
- ✔ Chạy test được trên nhiều hệ điều hành • ✔ Viết test được bằng C# – ngôn ngữ nhóm đã học
- ✔ Chạy test trong khi vẫn làm việc khác được
- ✗ **Gặp khó khăn khi xử lý cửa sổ upload file** – đây là hạn chế cần công cụ hỗ trợ bên ngoài

Sáu đặc điểm chính của Selenium

Mã nguồn mở

Sử dụng **miễn phí**, không cần chi trả chi phí bản quyền ⇒ giảm chi phí dự án, đặc biệt với dự án nhỏ hoặc môi trường học tập.

Cộng đồng hỗ trợ lớn

Dễ dàng tìm kiếm tài liệu và giải pháp khi gặp vấn đề. Tuy nhiên do có nhiều nguồn hướng dẫn khác nhau nên **cần biết chọn lọc giải pháp phù hợp**.

Hỗ trợ đa nền tảng

Hoạt động trên **Windows, macOS, Linux**, với khả năng **tái sử dụng kịch bản kiểm thử cao** giữa các nền tảng.

Chạy ở chế độ nền

Khi thực thi test script, người dùng **vẫn có thể làm việc khác** trên máy tính (đặc biệt ở chế độ headless) ⇒ tiết kiệm thời gian và tài nguyên.

Chỉ hỗ trợ trình duyệt

Selenium chủ yếu dùng để kiểm thử **ứng dụng web** và **không hỗ trợ kiểm thử ứng dụng desktop**.

Cần kết hợp công cụ khác

Với thao tác liên quan đến hệ điều hành như **upload/download file** hoặc ứng dụng bên ngoài, cần dùng thêm công cụ hỗ trợ (AutoIT, Robot class).

Ưu điểm và hạn chế của Selenium

ƯU ĐIỂM

- **Tự động hoá thao tác trình duyệt** – tester không cần nhập dữ liệu, nhấn nút hay kiểm tra kết quả thủ công
- **Tái sử dụng cao** – kịch bản xây dựng một lần, dùng lại nhiều lần ⇒ tiết kiệm đáng kể thời gian và công sức
- **Đa trình duyệt** – Chrome, Firefox, Safari, Edge ⇒ bảo đảm ứng dụng web hoạt động ổn định trên nhiều môi trường
- **Đa ngôn ngữ lập trình** – Java, Python, C#, Ruby ⇒ chọn được ngôn ngữ phù hợp với dự án và dễ tích hợp với công cụ khác

HẠN CHẾ

- **Phụ thuộc trình duyệt** – khi trình duyệt cập nhật phiên bản mới, tester có thể phải cập nhật lại driver hoặc thư viện để bảo đảm tương thích
- **Không hỗ trợ đầy đủ ứng dụng di động** – trường hợp cần kiểm thử mobile phải dùng công cụ khác như **Appium**
- **Khó với tình huống phức tạp** – xử lý nhiều cửa sổ, iframe, upload file đòi hỏi kỹ năng cao hơn từ tester
- **Không có báo cáo sẵn** – cần tích hợp thêm NUnit, TestNG, Allure để tạo báo cáo kiểm thử

THỜI LƯỢNG: ≈ 0,25 TIẾT

6.2

Các thành phần của bộ công cụ Selenium

- Selenium IDE
- Selenium WebDriver
- Selenium Grid
- Selenium RC (đã lỗi thời)

Bốn thành phần chính của Selenium Suite

Selenium IDE

Ghi và phát lại (record & playback).

Tiện ích mở rộng trên trình duyệt.

Không cần biết lập trình.

⇒ Dành cho người mới bắt đầu và tạo bản mẫu kịch bản nhanh.

Selenium WebDriver

Lập trình kiểm thử.

Thư viện điều khiển trình duyệt trực tiếp bằng mã lệnh.

⇒ Thành phần **mạnh nhất và được dùng nhiều nhất** trong dự án thực tế.

Selenium Grid

Kiểm thử song song.

Chạy nhiều kịch bản trên nhiều máy, nhiều hệ điều hành và nhiều trình duyệt **cùng lúc**.

⇒ Rút ngắn mạnh thời gian kiểm thử.

Selenium RC (Remote Control)

Tiền thân của WebDriver.

Điều khiển trình duyệt gián tiếp qua JavaScript.

⇒ **Đã lỗi thời**, không còn được khuyến nghị sử dụng.

THỜI LƯỢNG: ≈ 1,25 TIẾT

6.3

Selenium IDE

- Cài đặt
- Giao diện
- Quy trình ghi và phát lại
- Các lệnh thông dụng
- Lợi ích và hạn chế

Selenium IDE – khái niệm và phạm vi sử dụng

KHÁI NIỆM

- **Selenium IDE** (Integrated Development Environment) là **tiện ích mở rộng trên trình duyệt** (Firefox, Chrome) cho phép **ghi lại (record)** và **phát lại (playback)** thao tác của người dùng
- Mọi thao tác – nhấp chuột, nhập văn bản, chuyển trang – được ghi lại thành **kịch bản kiểm thử**
- Người dùng có thể **chỉnh sửa, lưu trữ và chạy lại** kịch bản khi cần tái kiểm thử
- **Không yêu cầu kiến thức lập trình**

PHẠM VI PHÙ HỢP

- **Ưu điểm lớn nhất: tốc độ triển khai** – kịch bản được tạo gần như tức thì ngay sau thao tác
- Phù hợp với: **kiểm thử đơn vị đơn giản, kiểm thử khói (smoke testing)**, tạo **bản mẫu kịch bản** trước khi chuyển sang WebDriver
- **Hạn chế:** khó xử lý logic phức tạp, điều kiện rẽ nhánh sâu, đối tượng giao diện có tính động cao
- ⇒ Chỉ nên dùng cho **dự án nhỏ hoặc giai đoạn thử nghiệm ban đầu**

Cài đặt Selenium IDE trên trình duyệt Firefox

BƯỚC 1

Bước 1

Mở trình duyệt **Firefox**.

BƯỚC 2

Bước 2

Mở trang **Add-ons** bằng tổ hợp phím **Ctrl + Shift + A**.

BƯỚC 3

Bước 3

Tìm kiếm tiện ích **"Selenium IDE"** trong kho tiện ích.

BƯỚC 4

Bước 4

Chọn Selenium IDE và nhấn **Add to Firefox** để cài đặt.

BƯỚC 5

Bước 5

Nhấp biểu tượng Selenium IDE, chọn **Create a new project** và đặt tên dự án.

Giao diện Selenium IDE – các khu vực chức năng

① Thanh tiêu đề & quản lý dự án — Demo Project | New · Open · Save

② Thanh công cụ (Toolbar) — ▶ Run all tests · ▶ Run current test · ● Record · ⏸ Pause · ■ Stop · ⚙ Tốc độ chạy

③ Trình quản lý dự án

- Danh sách Test Case
- Nhóm thành Test Suite
- Đổi tên, chọn test để thực thi

VD: TC01_Login_Success

④ Bảng lệnh kiểm thử (Command Table)

Command	Target	Value
open	/dang-nhap	
type	id=txtUsername	sv001
type	id=txtPassword	Abc@1234
click	id=btnLogin	
assertText	css=.welcome	Xin chào sv001

⑤ Khu vực Log & Debug (Console) — ✔ open OK (0.4s) ✔ type OK ✔ click OK ✗ assertText FAILED: Element not found

Quy trình ghi (Record) và phát lại (Playback)

BƯỚC 1

① Tạo dự án mới

Nhấn Create a new project và đặt tên cho dự án.

BƯỚC 2

② Thêm test case

Nhấn dấu **"+"** trên thanh kiểm thử để thêm trường hợp kiểm thử mới.

BƯỚC 3

③ Chỉ định URL cơ sở

Xác định nơi bài kiểm thử bắt đầu, ví dụ `https://www.thegioididong.com/`.

BƯỚC 4

④ Bắt đầu ghi

Nhấn nút **Record**; mọi thao tác trên trình duyệt sẽ được ghi lại thành lệnh.

BƯỚC 5

⑤ Thao tác & dừng ghi

Thực hiện các thao tác cần kiểm thử, sau đó **tạm dừng ghi** – tất cả lệnh đã được lưu.

BƯỚC 6

⑥ Phát lại & đánh giá

Nhấn **Play** để chạy lại; xem bản tóm tắt để xác định bước nào **Pass** hay **Fail**.

Các lệnh thông dụng trong Selenium IDE

Lệnh (Command)	Mục đích	Cú pháp / ví dụ
open	Mở một URL cụ thể trên trình duyệt	Command: `open` Target: `/dang-nhap`
click	Nhấp vào một phần tử trên trang web	Command: `click` Target: `id=btnLogin`
type	Nhập dữ liệu vào một trường input	Command: `type` Target: `id=txtUsername` • Value: `sv001`
assertText	Kiểm tra nội dung văn bản – **dừng test nếu sai**	Command: `assertText` Target: `css=.welcome` • Value: `Xin chào sv001`
verifyText	Kiểm tra nội dung – **ghi nhận lỗi nhưng chạy tiếp**	Command: `verifyText` Target: `id=footer` • Value: `© 2026`
waitForElementPresent	Chờ cho đến khi phần tử xuất hiện	Target: `id=result` • Value: `5000` (ms)

Mỗi bước kiểm thử gồm 3 thành phần: Command ("Làm gì?") – **Target** ("Làm vào đâu?") – **Value** ("Dữ liệu là gì?").

Assert và Verify – hai lệnh kiểm chứng quan trọng

ASSERT – Khẳng định

- **Dừng test ngay nếu sai**
- Dùng cho các **điều kiện tiên quyết**: nếu bước này sai thì các bước sau không còn ý nghĩa
- **Ví dụ:** nếu **không đăng nhập thành công** thì không cần chạy các bước kiểm tra giỏ hàng, thanh toán phía sau
- Giúp **tiết kiệm thời gian** chạy test và tránh hàng loạt lỗi giả

VERIFY – Xác minh

- **Ghi nhận lỗi nhưng vẫn chạy tiếp**
- Dùng khi muốn kiểm tra **nhiều chi tiết độc lập** trên cùng một trang
- **Ví dụ:** kiểm tra logo, banner, menu và chân trang có hiển thị đủ hay không – nếu logo sai vẫn nên kiểm tra tiếp các phần còn lại
- Giúp **thu thập được nhiều lỗi** trong một lần chạy

Lợi ích của Selenium IDE

Mã nguồn mở & tiết kiệm chi phí

Miễn phí cho mọi người dùng, không cần trả phí bản quyền đắt đỏ.

Tương thích đa nền tảng

Hỗ trợ nhiều hệ điều hành và trình duyệt khác nhau.

Tự động hoá dễ tiếp cận

Người có kỹ năng lập trình hạn chế vẫn xây dựng được kịch bản kiểm thử.

Xuất kịch bản linh hoạt

Cho phép **Export** test script sang **Java, C#, Python** để chuyển sang WebDriver.

Gỡ lỗi nhanh

Công cụ debug tích hợp giúp xác định và khắc phục lỗi ngay trong quá trình kiểm thử.

Điều khiển tốc độ chạy

Chạy nhanh cho kiểm thử hồi quy; **chạy chậm** rất hữu ích khi giảng dạy để sinh viên quan sát kịp thao tác.

Những hạn chế của Selenium IDE

- **Chỉ hỗ trợ ứng dụng web trên trình duyệt** – không dùng được cho ứng dụng máy tính để bàn (desktop)
- **Không hỗ trợ kiểm thử dịch vụ web** như REST hoặc SOAP ⇒ không phù hợp cho bài toán kiểm thử API
- **Hạn chế về khả năng mở rộng và bảo trì** – kịch bản tạo bằng cơ chế ghi/phát lại nên khi giao diện thay đổi, tester phải chỉnh sửa lại nhiều bước, khó áp dụng cho hệ thống lớn
- **Không có sẵn chức năng báo cáo kết quả** – cần kết hợp với công cụ hoặc framework khác để theo dõi và quản lý kết quả
- **Không hỗ trợ kiểm thử hình ảnh trực tiếp** – cần tích hợp công cụ bên ngoài nếu muốn so sánh giao diện dựa trên hình ảnh
- **Hỗ trợ chủ yếu từ cộng đồng** – có thể gây khó khăn cho người mới khi gặp vấn đề phức tạp chưa có hướng dẫn rõ ràng

THỜI LƯỢNG: ≈ 0,5 TIẾT

6.4

Selenium Grid

- Đặc điểm chính
- Kiến trúc Hub – Node
- So sánh tuần tự và song song

Bốn đặc tính kỹ thuật cốt lõi của Selenium Grid

① Thực thi song song

Thay vì xếp hàng chờ từng kịch bản chạy tuần tự, Grid **kích hoạt đồng thời hàng loạt bài kiểm tra**, giảm thời gian kiểm thử từ **vài tiếng xuống vài phút**.

② Tính đa môi trường

Cấu hình mạng lưới gồm nhiều hệ điều hành (Windows, Linux, macOS) và nhiều trình duyệt (Chrome, Firefox, Safari, Edge) ⇒ **kiểm duyệt độ tương thích toàn diện**.

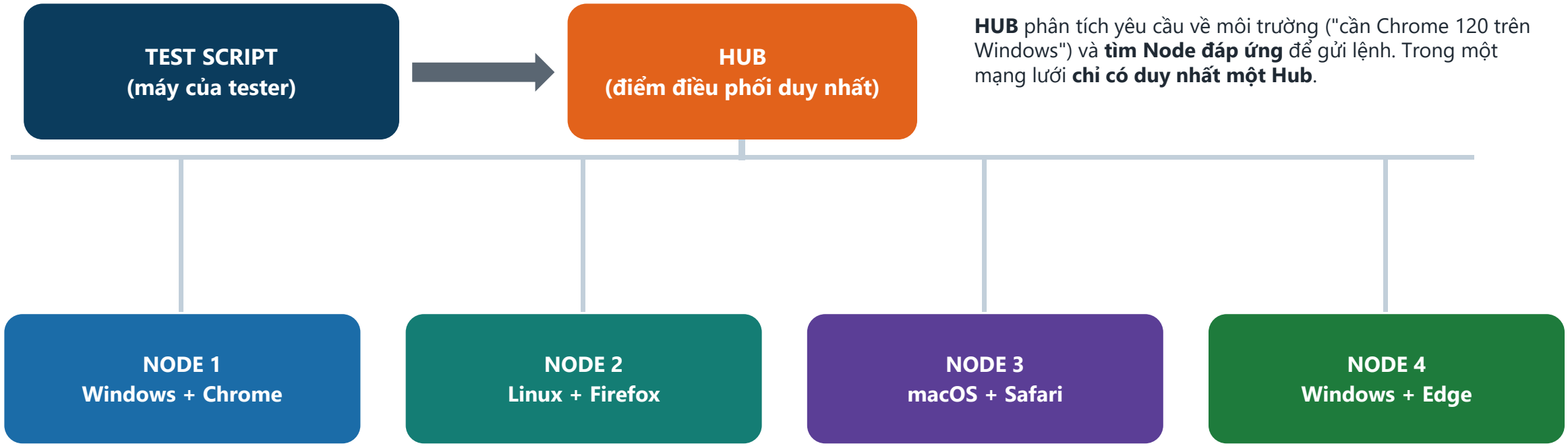
③ Quản lý tập trung

Dù có hàng chục hay hàng trăm máy trạm, kỹ sư kiểm thử **chỉ gửi yêu cầu đến một điểm điều phối duy nhất là Hub** ⇒ loại bỏ việc cấu hình thủ công từng máy.

④ Tối ưu tài nguyên

Thay vì dồn ép một máy tính đơn lẻ xử lý quá tải, Grid **phân tán khối lượng công việc** trên nhiều máy trong mạng nội bộ hoặc trên cloud ⇒ hệ sinh thái kiểm thử bền vững, dễ mở rộng.

Kiến trúc Hub – Node của Selenium Grid



HUB phân tích yêu cầu về môi trường ("cần Chrome 120 trên Windows") và **tìm Node đáp ứng** để gửi lệnh. Trong một mạng lưới **chỉ có duy nhất một Hub**.

NODE là các máy trạm thực tế (máy vật lý hoặc máy ảo) đã đăng ký với Hub, chứa trình duyệt và driver tương ứng. Khi nhận lệnh từ Hub, Node **trực tiếp khởi động trình duyệt và thực thi các bước kiểm thử**.

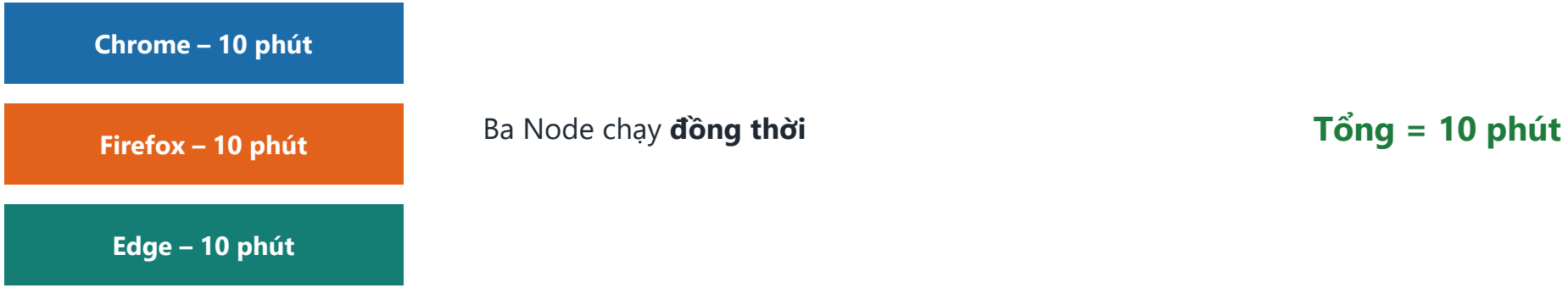
So sánh thực thi tuần tự và thực thi song song

Kiểm thử tương thích một website trên ba trình duyệt Chrome, Firefox và Edge (mỗi trình duyệt 10 phút)

THỰC THI TUẦN TỰ (không dùng Grid)



THỰC THI SONG SONG (dùng Selenium Grid)



Khi chạy song song, **tổng thời gian chỉ phụ thuộc vào trình duyệt chậm nhất**, không còn là tổng thời gian của cả ba. Thực thi tuần tự gây **nghẽn trong quy trình phát triển** và làm chậm việc phản hồi lỗi.

THỜI LƯỢNG: ≈ 1 TIẾT

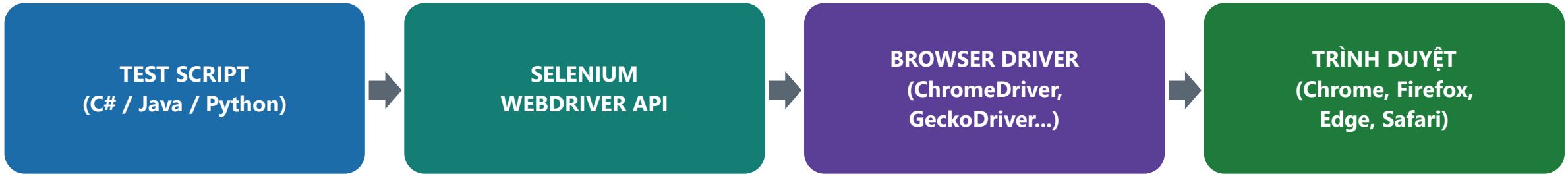
6.5

Selenium WebDriver

- Khái niệm và kiến trúc
- Khi nào nên dùng
- Session – Element – Locator – Wait – Alert
- Ưu điểm và nhược điểm

Selenium WebDriver – khái niệm và kiến trúc

Selenium WebDriver là **thư viện mã nguồn mở, miễn phí** hỗ trợ kiểm thử tự động trên website. WebDriver **cần kết hợp với một ngôn ngữ lập trình** để vận hành.



ĐIỂM VƯỢT TRỘI SO VỚI SELENIUM RC

- WebDriver **không dựa vào JavaScript** để tự động hoá, mà **giao tiếp trực tiếp với trình duyệt** thông qua driver tương ứng
- Cách tiếp cận **hiện đại và ổn định hơn**, mô phỏng hành vi người dùng thật chính xác hơn
- Hỗ trợ cùng bộ ngôn ngữ như RC: **Java, C#, PHP, Python, Ruby...**

Khi nào nên và không nên sử dụng WebDriver

NÊN SỬ DỤNG KHI...

- **Kiểm thử hồi quy** – ứng dụng thay đổi, cần chạy lại bộ test cũ để bảo đảm chức năng hiện tại không bị ảnh hưởng
- **Kiểm thử trình duyệt chéo** – cần chạy trên Chrome, Firefox, Edge, Safari để bảo đảm tính tương thích
- **Tự động hoá tác vụ lặp lại** – nhập liệu, click, xử lý biểu mẫu nhiều lần
- **Dự án quy mô lớn, dài hạn** – WebDriver mang lại lợi ích cao nhất trong việc bảo trì và mở rộng bộ test theo thời gian
- **Tích hợp CI/CD** – tự động chạy test sau mỗi lần build để phát hiện lỗi sớm

KHÔNG NÊN / HẠN CHẾ DÙNG KHI...

- **Dự án nhỏ, ngắn hạn** – chi phí viết và bảo trì script cao hơn lợi ích mang lại
- **Ứng dụng mobile native** – WebDriver tập trung vào trình duyệt; dùng được với mobile web nhưng **không tối ưu cho app native** (nên dùng Appium)
- **Giao diện thay đổi liên tục** – nếu UI chưa ổn định, việc bảo trì các **locator** sẽ tốn rất nhiều thời gian
- **Kiểm thử API thuần tuý** – nên dùng Postman hoặc thư viện HTTP chuyên dụng

Năm khái niệm cần biết trong Selenium WebDriver

Khái niệm	Định nghĩa và vai trò
Session (Phiên làm việc)	Là **kết nối giữa WebDriver và trình duyệt**. Mỗi session có một **ID duy nhất** dùng để xác định các lệnh và kết quả. Session **được tạo** khi khởi tạo đối tượng WebDriver, và **kết thúc** khi đóng trình duyệt hoặc gọi phương thức `Quit()`.
Element (Phần tử)	Là đối tượng đại diện cho **một thành phần giao diện** trên trang web: nút bấm, liên kết, ô nhập liệu... Tìm bằng `FindElement()` (một phần tử) hoặc `FindElements()` (nhiều phần tử). Tương tác qua: `Click()`, `SendKeys()`, `Clear()`, `Text`, `GetAttribute()`.
Locator (Bộ định vị)	Là **cách xác định vị trí** của một Element trên trang, dựa vào thuộc tính hoặc biểu thức. Các loại: **ID, Name, ClassName, TagName, LinkText, CssSelector, XPath**. Locator được truyền vào `FindElement()` để tìm phần tử mong muốn.
Wait (Chờ đợi)	Kỹ thuật cho phép **đợi một khoảng thời gian hoặc đợi đến khi một điều kiện được thoả mãn** trước khi thực hiện lệnh tiếp theo. **Implicit Wait:** cài đặt **toàn cục**, đặt thời gian tối đa WebDriver chờ cho **mỗi lần** tìm Element. **Explicit Wait:** cài đặt **cục bộ**, đặt thời gian tối đa **và một điều kiện cụ thể** cho một Element nhất định.
Alert (Hộp thoại)	Là hộp thoại xuất hiện trên trang web, yêu cầu người dùng nhập liệu hoặc xác nhận hành động. Ba loại: **Simple Alert, Confirmation Alert, Prompt Alert**. Xử lý bằng lớp `Alert` với các phương thức `Accept()`, `Dismiss()`, `SendKeys()`.

Ưu điểm và nhược điểm của Selenium WebDriver

ƯU ĐIỂM

- **Mã nguồn mở, miễn phí** – không mất chi phí bản quyền, cộng đồng hỗ trợ lớn
- **Đa nền tảng** – Windows, Linux, macOS
- **Đa ngôn ngữ** – Java, Python, C#, PHP, Ruby, JavaScript
- **Đa trình duyệt** – Chrome, Firefox, IE, Safari, Edge
- **Tương tác trực tiếp** – gửi lệnh thẳng đến trình duyệt, giả lập hành vi người dùng thật tốt hơn các phiên bản trước
- **Tích hợp CI/CD** – Jenkins, Docker, Azure DevOps
- **Kiểm thử song song** – kết hợp Selenium Grid

NHƯỢC ĐIỂM

- **Yêu cầu kỹ năng lập trình** – tester cần biết viết mã, không phù hợp với người chỉ làm kiểm thử thủ công
- **Chỉ hỗ trợ Web** – không kiểm thử được ứng dụng desktop (phải kết hợp công cụ khác)
- **Bảo trì script tốn kém** – khi giao diện website thay đổi thường xuyên, script dễ bị lỗi và mất thời gian cập nhật
- **Không có báo cáo sẵn** – cần tích hợp thêm NUnit, TestNG hoặc JUnit để tạo báo cáo
- **Hỗ trợ kỹ thuật hạn chế** – là mã nguồn mở nên không có hỗ trợ chuyên biệt như công cụ thương mại

THỜI LƯỢNG: ≈ 0,75 TIẾT

6.6

Các thao tác cơ bản trong Selenium WebDriver (C#)

- Khởi tạo trình duyệt
- Điều hướng
- Định vị phần tử
- Tương tác
- Thao tác đặc biệt
- Đóng trình duyệt

① Khởi tạo và thiết lập trình duyệt

Hai thư viện bắt buộc: OpenQA.Selenium và OpenQA.Selenium.Chrome.

```
using OpenQA.Selenium;
using OpenQA.Selenium.Chrome;

// Khoi tao Chrome Driver
IWebDriver driver = new ChromeDriver();

// Mo rong cua so trinh duyet toi da
driver.Manage().Window.Maximize();
```

② Điều hướng (Navigation)


```
// Mo mot URL
driver.Navigate().GoToUrl("https://www.google.com");
// hoac
driver.Url = "https://www.google.com";

driver.Navigate().Back();      // Quay lai trang truoc
driver.Navigate().Forward();   // Tien toi trang sau
driver.Navigate().Refresh();   // Tai lai trang
```

③ Định vị phần tử (Locators) – dùng lớp By


```
driver.FindElement(By.Id("elementId"));            // theo thuoc tinh id
driver.FindElement(By.Name("elementName"));        // theo thuoc tinh name
driver.FindElement(By.ClassName("className"));     // theo thuoc tinh class
driver.FindElement(By.TagName("input"));           // theo the HTML
driver.FindElement(By.LinkText("Dang nhap"));      // theo van ban lien ket
driver.FindElement(By.CssSelector("#login .btn")); // theo CSS selector
driver.FindElement(By.XPath("//button[@id='btnLogin']")); // theo XPath
```

Thứ tự ưu tiên khuyến nghị: `Id` → `Name` → `CssSelector` → `XPath`. Ưu tiên **Id** vì nhanh nhất và ổn định nhất; hạn chế XPath tuyệt đối (dạng `/html/body/div[3]/...`) vì rất dễ hỏng khi giao diện thay đổi.

④ Tương tác với phần tử (WebElement)


```
IWebElement element = driver.FindElement(By.Id("elementId"));

element.Click();                    // Nhap chuot
element.SendKeys("text");           // Nhap du lieu
element.Clear();                    // Xoa du lieu trong o nhap
string text = element.Text;         // Lay noi dung van ban
bool visible = element.Displayed;   // Kiem tra co hien thi khong
string val = element.GetAttribute("value");  // Lay gia tri thuoc tinh
```

⑤ Thao tác đặc biệt và ⑥ Đóng trình duyệt


```
using OpenQA.Selenium.Support.UI;

// Xu ly Dropdown
var select = new SelectElement(driver.FindElement(By.Id("ddlCity")));
select.SelectByText("Vinh Long");

// Chuyen vao Iframe
driver.SwitchTo().Frame("frameName");

// Explicit Wait - cho den khi phan tu xuat hien (toi da 10 giay)
var wait = new WebDriverWait(driver, TimeSpan.FromSeconds(10));
wait.Until(drv => drv.FindElement(By.Id("result")).Displayed);

driver.Close();   // Dong tab hien tai
driver.Quit();    // Dong toan bo trinh duyet va giai phong bo nho
```

Luôn dùng `Quit()` ở cuối test để giải phóng tiến trình driver, tránh treo tài nguyên máy khi chạy hàng loạt test.

Ví dụ hoàn chỉnh: kịch bản kiểm thử chức năng đăng nhập


```
using OpenQA.Selenium;
using OpenQA.Selenium.Chrome;

IWebDriver driver = new ChromeDriver();
driver.Manage().Window.Maximize();

driver.Navigate().GoToUrl("https://the-internet.herokuapp.com/login");

driver.FindElement(By.Id("username")).SendKeys("tomsmith");
driver.FindElement(By.Id("password")).SendKeys("SuperSecretPassword!");
driver.FindElement(By.CssSelector("button.radius")).Click();

// Kiem chung ket qua
string msg = driver.FindElement(By.Id("flash")).Text;
Console.WriteLine(msg.Contains("You logged into a secure area")
                  ? "TEST PASSED" : "TEST FAILED");

driver.Quit();
```

Quy trình xây dựng kịch bản kiểm thử với Selenium

BƯỚC 1

① Xác định phạm vi & mục tiêu

Xác định chức năng cần tự động hoá, dựa trên tài liệu yêu cầu hoặc test case thủ công.

BƯỚC 2

② Lựa chọn công cụ & môi trường

Chọn trình duyệt, ngôn ngữ (C#), IDE (Visual Studio); cài ChromeDriver/Gecko Driver.

BƯỚC 3

③ Xây dựng test script

Mở trình duyệt → truy cập trang → thao tác (nhập, click) → **kiểm tra kết quả**.

BƯỚC 4

④ Thực thi kịch bản

Chạy script, quan sát các bước có thực hiện đúng không, có lỗi phát sinh không.

BƯỚC 5

⑤ Phân tích & đánh giá kết quả

So sánh kết quả thực tế với mong đợi để xác định Pass/Fail; tìm nguyên nhân nếu lỗi.

BƯỚC 6

⑥ Bảo trì & cập nhật script

Khi giao diện hoặc chức năng thay đổi, cập nhật lại script để bảo đảm tính chính xác.

Ví dụ áp dụng – kiểm thử chức năng đăng nhập: ① xác định yêu cầu đăng nhập với tài khoản hợp lệ → ② chọn C# + Selenium WebDriver → ③ viết script nhập username, password, click login → ④ chạy test → ⑤ kiểm tra kết quả hiển thị → ⑥ sửa nếu có lỗi.

THỜI LƯỢNG: ≈ 1,5 TIẾT

6.8

Bài tập vận dụng và câu hỏi ôn tập

- 5 bài tập
- 10 bài tập tự luyện
- 6 câu hỏi ôn tập

Phân tích tình huống lựa chọn công cụ Selenium

- Một công ty cần kiểm thử **website bán hàng** trên các trình duyệt **Chrome, Firefox và Edge** sau **mỗi lần cập nhật hệ thống**.
- **Yêu cầu:**
  - a) Xác định nên sử dụng **Selenium IDE, Selenium WebDriver hay Selenium Grid**.tr10
  - b) Giải thích lý do lựa chọn.
  - c) Mô tả ngắn gọn quy trình triển khai kiểm thử.

Thiết kế test case cho chức năng đăng nhập

- Cho chức năng **Đăng nhập** của một website với hai ô nhập liệu **Username**, **Password** và nút **Login**.
- **Yêu cầu:**
  - a) Xây dựng tối thiểu **05 test case** cho chức năng đăng nhập.
  - b) Xác định dữ liệu đầu vào và kết quả mong đợi cho từng test case.
  - c) Cho biết những test case nào **phù hợp để tự động hoá** bằng Selenium và giải thích.tr12tr19

Phân tích Locator trong Selenium

- Cho đoạn mã HTML sau:
  - `<input id="txtUsername" name="username" class="form-control">`
  - `<button id="btnLogin">Login</button>`
- **Yêu cầu:** Viết các câu lệnh Selenium WebDriver (C#) để xác định phần tử bằng các Locator sau:
  - `By.Id` • `By.Name` • `By.ClassName` • `By.CssSelector` • `By.XPath`tr32
- **Mở rộng:** Sắp xếp các locator trên theo thứ tự ưu tiên nên dùng và giải thích.tr32

Xây dựng quy trình kiểm thử với Selenium

- Một tester cần tự động kiểm thử chức năng **tìm kiếm sản phẩm** trên website thương mại điện tử.
- **Yêu cầu:** Trình bày các bước của quy trình xây dựng kịch bản kiểm thử, mỗi bước nêu rõ công việc phải thực hiện:
  - Xác định mục tiêu • Chuẩn bị môi trường • Viết script
  - Thực thi • Phân tích kết quả • Bảo trì script

Viết chương trình Selenium WebDriver cơ bản (C#)

- Viết chương trình Selenium WebDriver bằng C# thực hiện các thao tác sau:
  - ① Khởi tạo ChromeDriver.
  - ② Mở website `https://the-internet.herokuapp.com/login`.
  - ③ Nhập Username: `tomsmith` và Password: `SuperSecretPassword!`.
  - ④ Nhấn nút **Login**.
  - ⑤ Kiểm tra thông báo đăng nhập thành công.
  - ⑥ Đóng toàn bộ trình duyệt bằng `Quit()`.
- **Yêu cầu mở rộng:** Xác định các Locator được sử dụng và giải thích vai trò của từng câu lệnh.tr36

Bộ 10 bài tập tự luyện (Chương 6)

- **Bài 1.** So sánh Selenium IDE, WebDriver và Grid theo 5 tiêu chí. Nêu tình huống sử dụng phù hợp cho từng thành phần.tr12 tr26
- **Bài 2.** Phân biệt `Close()` và `Quit()`. Điều gì xảy ra nếu chỉ dùng `Close()` khi chạy 100 test case liên tiếp?
- **Bài 3.** Phân biệt **Implicit Wait** và **Explicit Wait**.tr27 Vì sao không nên dùng `Thread.Sleep()`?tr45
- **Bài 4.** Cho HTML: `<a href="/gio-hang" class="nav-link cart">Giỏ hàng (3)</a>`. Viết 4 cách định vị phần tử này bằng C#.tr32
- **Bài 5.** Giải thích vì sao `By.ClassName("btn")` là locator rủi ro. Đề xuất cách thay thế an toàn hơn.
- **Bài 6.** Viết script C# xử lý một **Confirmation Alert**: nhấn nút "Xoá", xác nhận alert và kiểm tra thông báo kết quả.
- **Bài 7.** Một Grid có 1 Hub và 4 Node. Bộ test có 40 test case, mỗi test 3 phút. Tính thời gian chạy tuần tự và song song.
- **Bài 8.** Viết script C# chọn giá trị "Vĩnh Long" trong dropdown `<select id="ddlCity">` bằng 3 cách khác nhau.
- **Bài 9.** Trình bày cách tổ chức một dự án Selenium theo **Page Object Model** cho website có 4 trang. Nêu cấu trúc thư mục.
- **Bài 10.** Test đăng nhập chạy tốt trên máy tester nhưng fail trên CI server. Nêu 5 nguyên nhân có thể và cách khắc phục.

Câu hỏi ôn tập chương 6

1. Trình bày khái niệm **Selenium** và phân tích các đặc điểm nổi bật của Selenium trong kiểm thử tự động.tr5/tr8

2. Phân biệt chức năng của **Selenium IDE** tr12**, Selenium WebDriver** tr 25 **và Selenium Grid**. Tr21 Cho ví dụ tình huống sử dụng phù hợp với từng thành phần.

3. Trình bày các khái niệm quan trọng trong Selenium WebDriver: **Session, Element, Locator, Wait và Alert** tr27 Giải thích vai trò của từng khái niệm.

4. Phân tích **quy trình xây dựng một kịch bản kiểm thử tự động** bằng Selenium. Tr5 Theo em bước nào quan trọng nhất? Giải thích lý do.

5. Phân tích ưu điểm và hạn chế của **Selenium WebDriver** tr28 **so với Selenium IDE** tr12. Khi nào nên chọn IDE và khi nào nên dùng WebDriver?

6. Trình bày kiến trúc **Hub – Node** tr22 của Selenium Grid. Vì sao thực thi song song giúp rút ngắn đáng kể thời gian kiểm thử?

Tóm tắt chương 6

- **Selenium** là bộ công cụ kiểm thử tự động mã nguồn mở cho ứng dụng web, hỗ trợ đa trình duyệt, đa nền tảng và đa ngôn ngữ lập trình.
- **Selenium IDE** phù hợp cho người mới và tạo bản mẫu nhanh; **WebDriver** là thành phần chính cho dự án thực tế; **Grid** dùng để chạy song song đa môi trường.
- Năm khái niệm cốt lõi của WebDriver: **Session, Element, Locator, Wait, Alert** – nắm vững chúng là nắm được cách viết mọi kịch bản Selenium.
- Thứ tự ưu tiên locator: **Id** → **Name** → **CssSelector** → **XPath**; tránh ClassName; tốt nhất là phối hợp với lập trình viên thêm thuộc tính `data-testid`.
- Luôn dùng **Explicit Wait** thay cho `Thread.Sleep`, và luôn gọi **`Quit()`** ở cuối test để giải phóng tài nguyên.
- Áp dụng **Page Object Model** ngay từ đầu là cách hiệu quả nhất để giảm chi phí bảo trì – vốn là điểm yếu lớn nhất của kiểm thử tự động giao diện.

Đánh giá kết quả học tập – Selenium

BÀI QUIZ • 30%

- **15 câu trắc nghiệm**, thời gian 15 phút
- Làm vào **đầu buổi chiều** ngày 1, 2, 3
- Mỗi câu **0,67 điểm** – làm tròn thang 10
- Chấm chéo tại lớp, công bố đáp án ngay
- Lấy **trung bình 3 bài quiz**

BÀI KIỂM TRA VIẾT 45 PHÚT • 70%

- **20 câu trắc nghiệm + 1 bài tập vận dụng**
- Làm vào **đầu ngày 4**, phủ cả 6 chương
- Trắc nghiệm **5 điểm** – bài tập **5 điểm**
- Mỗi chương góp khoảng **3–4 câu** trắc nghiệm
- Bài tập vận dụng chọn **1 trong 6 đề** của các chương

CÁCH TÍNH ĐIỂM QUÁ TRÌNH

Điểm quá trình = **0,7 × điểm bài kiểm tra viết 45 phút** + **0,3 × trung bình 3 bài quiz**

Ví dụ: bài viết 8,0 và ba quiz lần lượt 7,0 – 8,0 – 9,0 (trung bình 8,0) → điểm quá trình = 0,7 × 8,0 + 0,3 × 8,0 = **8,0**

Quiz 15 câu – Chương 6 (phần 1/3)

Câu 1. Selenium là công cụ tự động hoá dành cho:
A. Ứng dụng desktop B. Trình duyệt web C. Ứng dụng Android D. Kiểm thử API

Câu 2. Thành phần nào của Selenium dùng để GHI và PHÁT LẠI thao tác, không cần lập trình?
A. WebDriver B. Selenium IDE C. Selenium Grid D. Selenium RC

Câu 3. Thành phần nào dùng để chạy test SONG SONG trên nhiều máy, nhiều trình duyệt?
A. Selenium IDE B. WebDriver C. Selenium Grid D. Selenium RC

Câu 4. Trong kiến trúc Selenium Grid, thành phần nhận yêu cầu và phân phối công việc là:
A. Node B. Hub C. Driver D. Browser

Câu 5. Locator NÊN ƯU TIÊN dùng nhất là:
A. XPath tuyệt đối B. ClassName C. Id D. TagName

Quiz 15 câu – Chương 6 (phần 2/3)

Câu 6. Vì sao KHÔNG nên dùng XPath tuyệt đối dạng /html/body/div[3]/div[2]/input[1]?
A. Viết dài dòng B. Gãy ngay khi cấu trúc HTML thay đổi dù chức năng không đổi C. Không chạy được trên Chrome D. Chậm hơn Id 100 lần

Câu 7. Vì sao không nên định vị phần tử theo ClassName?
A. Selenium không hỗ trợ B. Class dùng cho trình bày nên hay đổi và thường không duy nhất C. Class chỉ có ở thẻ div D. Class chạy chậm nhất

Câu 8. Khác biệt giữa Assert và Verify là:
A. Assert nhanh hơn B. Assert dừng kịch bản khi sai, Verify ghi nhận rồi chạy tiếp C. Verify dừng kịch bản khi sai D. Không khác nhau

Câu 9. Ô nhập đang có sẵn chữ "abc", script gọi SendKeys("sv001"). Kết quả trong ô là:
A. sv001 B. abc C. abcsv001 D. Lỗi

Câu 10. driver.Close() khác driver.Quit() ở chỗ:
A. Close đóng tất cả cửa sổ B. Quit chỉ đóng cửa sổ hiện tại C. Close đóng cửa sổ hiện tại, Quit đóng tất cả và kết thúc phiên D. Không khác nhau

Quiz 15 câu – Chương 6 (phần 3/3)

Câu 11. Lỗi StaleElementReferenceException thường xảy ra khi:
A. Locator viết sai B. Dùng lại biến phần tử sau khi trang đã tải lại C. Trình duyệt quá cũ D. Quên gọi Clear()

Câu 12. Phần tử nhìn thấy rõ trên màn hình nhưng Selenium báo không tìm thấy. Nguyên nhân có thể là:
A. Trình duyệt bị lỗi B. Phần tử nằm trong iframe mà chưa chuyển vào iframe C. Máy tính thiếu RAM D. Chưa cài driver

Câu 13. Lệnh kiểm chứng (assertion) trong kiểm thử tự động do đâu cung cấp?
A. Selenium WebDriver B. Thư viện kiểm thử như NUnit, JUnit, pytest C. Trình duyệt D. Hệ điều hành

Câu 14. Một kịch bản kiểm thử tự động hoàn chỉnh gồm bốn khối theo thứ tự:
A. Hành động – Chuẩn bị – Dọn dẹp – Kiểm chứng B. Chuẩn bị – Hành động – Kiểm chứng – Dọn dẹp C. Kiểm chứng – Chuẩn bị – Hành động – Dọn dẹp D. Chuẩn bị – Kiểm chứng – Hành động – Dọn dẹp

Câu 15. Mẫu thiết kế Page Object Model giúp giải quyết vấn đề:
A. Test chạy chậm B. Phải sửa nhiều script khi giao diện thay đổi C. Không đọc được file Excel D. Không chạy song song được

Ngân hàng câu trắc nghiệm cho bài kiểm tra 45 phút – Chương 6

Câu 1. Cho HTML: **<input type="text" id="user" name="username" class="form-input" />** Locator BỀN NHẤT để định vị phần tử này là:
A. By.ClassName("form-input") B. By.Id("user") C. By.XPath("/html/body/div[2]/form/input[1]") D. By.TagName("input")

Câu 2. Một script bấm nút Đăng nhập rồi kiểm tra ngay chữ "Xin chào", thỉnh thoảng Fail dù hệ thống chạy đúng. Cách khắc phục ĐÚNG NHẤT là:
A. Chạy lại script khi Fail B. Thêm lệnh ngủ cố định 5 giây C. Dùng chờ tường minh đợi phần tử xuất hiện rồi mới kiểm chứng D. Bỏ bước kiểm chứng

Câu 3. Nhóm cần kiểm thử một chức năng với 50 bộ dữ liệu khác nhau, có đọc dữ liệu từ Excel, kết quả đưa vào pipeline CI. Thành phần Selenium phù hợp là:
A. Selenium IDE B. Selenium WebDriver C. Selenium RC D. Chỉ cần Grid

Câu 4. Test case kiểm tra "tìm kiếm từ khoá áo sơ mi". Cách kiểm chứng PHÙ HỢP NHẤT với dữ liệu thay đổi liên tục là:
A. So sánh với danh sách 24 sản phẩm cố định B. Chỉ kiểm tra trang không báo lỗi C. Kiểm tra mọi sản phẩm trả về đều chứa từ khoá trong tên D. Kiểm tra số lượng phải đúng bằng 24

Phân tích locator và viết kịch bản Selenium WebDriver

Tình huống. Trang đăng nhập có mã HTML rút gọn như sau:

```
<form id="loginForm">
  <input type="text" id="txtUser" name="username" class="inp" />
  <input type="password" name="password" class="inp" />
  <button class="btn btn-primary">Đăng nhập</button>
</form>
```

Đăng nhập thành công thì chuyển sang **/home** và hiển thị phần tử **Xin chào, …**

Yêu cầu.

a) Viết **hai cách định vị** cho mỗi phần tử: ô tên đăng nhập, ô mật khẩu, nút Đăng nhập. Chỉ rõ cách nào **bền hơn** và **giải thích**. (2,0 đ)

b) Viết **kịch bản kiểm thử** (mã giả hoặc C#) cho test case **"Đăng nhập thành công với tài khoản hợp lệ"**, đủ **bốn khối**: chuẩn bị – hành động – kiểm chứng – dọn dẹp. (2,0 đ)

c) Nêu **hai lỗi thường gặp** khiến kịch bản này chạy không ổn định và cách khắc phục. (1,0 đ)